

RDBMS & Database Programming With JDBC

Answer Key — UPI-Style Peer-to-Peer Wallet Assessment

SECTION A — Conceptual Understanding

Q1. Explain how Java runtime enables this wallet to work across operating systems.

Java source code is compiled by `javac` into platform-independent bytecode (`.class` files) instead of native machine code. This bytecode is executed by the Java Virtual Machine (JVM), and a separate JVM build exists for Windows, Linux, and macOS. Because the JVM — not the operating system — interprets/JIT-compiles the bytecode, the same compiled wallet application (`User.class`, `Wallet.class`, etc.) runs unmodified on any machine that has a JVM installed. This is the 'Write Once, Run Anywhere' (WORA) principle.

Logic:

- `javac` compiles `.java` → `.class` (bytecode), not CPU-specific instructions.
- The JVM is the OS-specific layer; it hides OS differences (file paths, threading, memory) behind a common API.
- JDBC follows the same idea: the wallet talks to `java.sql` interfaces, and a vendor-specific driver (MySQL/Postgres JAR) translates calls for that database — so the same Java code works with either database engine, on any OS.

Q2. Explain precision handling for money transfers.

Binary floating-point types (`float`, `double`) cannot represent many decimal fractions exactly (e.g., 0.1), so repeated additions/subtractions of amounts like `100.10` accumulate rounding errors — unacceptable in a financial system. The correct practice is to use `java.math.BigDecimal`, constructed from a `String` (new `BigDecimal("100.10")`) rather than a `double` literal, and to fix the scale (decimal places) and rounding mode explicitly with `setScale(2, RoundingMode.HALF_EVEN)`.

Logic:

- `BigDecimal` stores the unscaled value and scale internally, giving exact decimal arithmetic.
- Always compare `BigDecimal` values with `compareTo()`, never `equals()`, because `equals()` also checks scale (e.g., `2.0` vs `2.00` are 'equal' in value but not `equals()`-equal).
- In the database, the balance/amount columns should be `DECIMAL(12,2)` (or `NUMERIC`), matched by `PreparedStatement.setBigDecimal()` on the Java side — never store money as `FLOAT/DOUBLE` columns.

Q3. Explain transfer validation logic.

Before any transfer is committed, the application must validate the request in a strict order so that no partial or invalid transaction reaches the database: (1) input validity, (2) business-rule checks, (3) balance sufficiency, and only then (4) the actual debit/credit.

Logic:

- Null/format check: amount must not be null and must parse as a positive `BigDecimal`.
- Range check: amount > 0 (reject zero/negative transfers) and amount does not exceed any configured per-transaction limit.
- Identity check: sender UPI ID must not equal receiver UPI ID (no self-transfer), and the receiver must exist in the users table.
- Balance check: if `(amount.compareTo(sender.getBalance()) > 0)` throw new `InsufficientBalanceException(...)` — this is what stops the debit before it happens.

- Only after every check passes does the code enter the debit/credit block, ideally wrapped in a single SQL transaction (see Section C).

Q4. Explain loop usage for monthly summaries.

A monthly summary needs to iterate over every transaction row belonging to a user within a date range and aggregate values (total sent, total received, running balance). This is a natural fit for an enhanced for-each loop over a `List<Transaction>` (or a `ResultSet` while-loop when reading directly from JDBC), accumulating totals as it goes.

Logic:

- `while (resultSet.next()) { ... }` is the standard loop shape for walking a JDBC `ResultSet` row by row.
- Inside the loop, a running total (`BigDecimal totalSent = totalSent.add(amount)`) is updated per row — this is classic accumulator-pattern looping.
- A month filter (`WHERE txn_date BETWEEN ? AND ?`) reduces the rows fetched from the database, so the loop only iterates the relevant month instead of the full history — better performance than filtering in Java.

Q5. Explain why collections suit transaction storage.

Transaction storage needs a dynamically-growing, ordered list of records whose size is not known in advance (a user might have 3 transactions or 3,000). Arrays are fixed-size, so the Java Collections Framework — specifically `List<Transaction>` implemented as an `ArrayList` — is the natural in-memory structure: it grows automatically, preserves insertion/chronological order, and offers built-in iteration, sorting (`Collections.sort` with a `Comparator` on date), and searching.

Logic:

- `ArrayList<Transaction>` avoids manual array-resizing code and integrates with Java's Stream API for filtering (e.g., `transactions.stream().filter(t -> t.getStatus().equals("SUCCESS"))`).
- A `Map<String, User>` (keyed by UPI ID) gives $O(1)$ lookup of a user's profile instead of scanning a list linearly.
- Collections mirror the database result set in memory, so once data is fetched via JDBC it can be cached, re-sorted, or exported without hitting the database again.

Q6. Explain payment modes using polymorphism.

The wallet supports multiple ways to move money (bank transfer, wallet-to-wallet balance transfer, possibly UPI-collect later). Rather than writing separate if/else branches for each mode everywhere payment happens, a common `PaymentMethod` interface is defined with one method, `processPayment(double amount)`. `BankTransfer` and `WalletBalance` each implement it differently. Code that initiates a payment only ever calls `paymentMethod.processPayment(amount)` on a `PaymentMethod` reference — at runtime the JVM dispatches to whichever concrete class was actually instantiated. This is runtime (dynamic) polymorphism.

Logic:

- Declaring `PaymentMethod method = new BankTransfer();` vs `new WalletBalance();` lets the same calling code work unchanged for either mode.
- Adding a new mode later (e.g., `CreditCardPayment`) only requires a new class implementing the interface — no existing calling code changes (Open/Closed Principle).
- The correct overridden method is chosen at runtime based on the object's actual type, not its declared reference type.

SECTION B — Practical Tasks

QB1. User with Encapsulated Balance

A private double balance field cannot be touched from outside the class at all; the only doorway in or out is through `getBalance()/setBalance()`. This is the core encapsulation guarantee the task asks for.

```
public class User {
    private String name;
    private String upiId;
    private double balance; // private -> encapsulated

    public User(String name, String upiId) { // basic constructor
        this.name = name;
        this.upiId = upiId;
        this.balance = 0.0;
    }

    // Getters and setters control all access to balance
    public double getBalance() { return balance; }

    public void setBalance(double balance) {
        if (balance < 0) throw new IllegalArgumentException("Balance cannot be negative");
        this.balance = balance;
    }

    public String getName() { return name; }
    public String getUpiId() { return upiId; }
}
```

Logic:

- Marking balance private hides it from every other class, including subclasses in a different package.
- `setBalance()` is the single choke point where validation (e.g., 'no negative balance') is enforced, so invalid state can never be assigned directly.
- External code (e.g., `PaymentMethod` implementations) must call `user.setBalance(newAmount)` after computing the new value — it can never do `user.balance = x` directly.

QB2. Constructor Overloading

Two constructors share the same class name but differ in parameter list (arity/type) — the compiler picks the matching one based on the arguments supplied at the call site. `this(...)` is used to chain the simpler constructor into the more general one, avoiding duplicated initialization code.

```
public class User {
    private String name;
    private String upiId;
    private double balance;

    // Constructor 1: basic profile
    public User(String name, String upiId) {
        this(name, upiId, 0.0); // delegates to the other constructor
    }

    // Constructor 2: premium profile with initial balance
    public User(String name, String upiId, double initialBalance) {
        this.name = name;
        this.upiId = upiId;
        this.balance = initialBalance;
    }
}

// Usage:
User basicUser = new User("Riya", "riya@upi");
User premiumUser = new User("Aman", "aman@upi", 5000.00);
```

Logic:

- Overload resolution happens at compile time, based on the number/type of arguments — this is a form of compile-time (static) polymorphism, distinct from the runtime polymorphism used for PaymentMethod.
- Chaining with this(name, upId, 0.0) keeps a single source of truth for field assignment, so a future change (e.g., adding validation) only needs to be written once.

QB3. PaymentMethod Interface

The interface defines a single contract, processPayment(double amount), that every payment mode must honor. BankTransfer and WalletBalance each supply their own implementation of that method — different internal logic, same external signature — which is exactly what an interface is for.

```
public interface PaymentMethod {
    void processPayment(double amount) throws InsufficientBalanceException;
}

public class BankTransfer implements PaymentMethod {
    private User user;
    public BankTransfer(User user) { this.user = user; }

    @Override
    public void processPayment(double amount) throws InsufficientBalanceException {
        if (amount > user.getBalance())
            throw new InsufficientBalanceException("Bank transfer failed: insufficient funds");
        user.setBalance(user.getBalance() - amount);
        System.out.println("Bank transfer of Rs." + amount + " successful.");
    }
}

public class WalletBalance implements PaymentMethod {
    private User user;
    public WalletBalance(User user) { this.user = user; }

    @Override
    public void processPayment(double amount) throws InsufficientBalanceException {
        if (amount > user.getBalance())
            throw new InsufficientBalanceException("Wallet payment failed: insufficient balance");
        user.setBalance(user.getBalance() - amount);
        System.out.println("Wallet payment of Rs." + amount + " successful.");
    }
}

// Usage (runtime polymorphism):
PaymentMethod method = useBank ? new BankTransfer(user) : new WalletBalance(user);
method.processPayment(250.00);
```

Logic:

- Calling code depends only on the PaymentMethod type, not on the concrete class, so it stays unchanged if a new payment mode is added.
- Declaring throws InsufficientBalanceException on the interface method forces every implementation to either handle or propagate the same checked exception, keeping error handling consistent.

QB4. InsufficientBalanceException

Extending Exception (a checked exception) forces every caller to either catch it or declare it with throws, so an insufficient-balance case can never be silently ignored — the compiler enforces handling at every call site. The custom message carries the requested vs. available amount for clear diagnostics.

```
public class InsufficientBalanceException extends Exception {
    public InsufficientBalanceException(String message) {
        super(message);
    }
}
```

```
// Throwing logic (inside a transfer method):
public void debit(User user, double amount) throws InsufficientBalanceException {
    if (amount > user.getBalance()) {
        throw new InsufficientBalanceException(
            "Transaction of Rs." + amount + " exceeds available balance of Rs." + user.getBalance());
    }
    user.setBalance(user.getBalance() - amount);
}

// Calling code handles it:
try {
    debit(user, 10000.00);
} catch (InsufficientBalanceException e) {
    System.out.println("Transfer declined: " + e.getMessage());
}
```

Logic:

- The check `amount > user.getBalance()` happens strictly before `setBalance()` is called, so the balance is never mutated when funds are insufficient.
- Using a dedicated exception class (instead of a generic `RuntimeException`) lets calling code catch this specific failure and distinguish it from other errors (e.g., a database connection failure).

QB5. Mask UPI ID

`charAt(0)` grabs the first character to keep visible. `indexOf('@')` locates the boundary between the local part and the domain. `substring(atIndex)` extracts everything from '@' to the end untouched. The characters strictly between index 1 and the '@' are replaced with '*' in a loop, then the three pieces are concatenated.

```
public static String maskUpiId(String upiId) {
    int atIndex = upiId.indexOf('@');
    if (atIndex < 2) return upiId; // too short to mask meaningfully

    char first = upiId.charAt(0); // keep first character
    String domainPart = upiId.substring(atIndex); // "@upi" onward, kept as-is
    StringBuilder masked = new StringBuilder();
    masked.append(first);
    for (int i = 1; i < atIndex; i++) {
        masked.append('*'); // mask every char between first and '@'
    }
    masked.append(domainPart);
    return masked.toString();
}

// Example: maskUpiId("user@upi") -> "u***@upi"
```

Logic:

- `substring(atIndex)` is a lightweight way to preserve a suffix without manual character copying.
- The loop count (`atIndex - 1`) automatically adapts the number of asterisks to the local part's length, so 'user@upi' and 'ri@upi' are masked correctly without hardcoding '3 stars'.
- A guard (`atIndex < 2`) avoids an `ArrayIndexOutOfBoundsException`-style edge case on abnormally short IDs.

SECTION C — Mini Project

QC1. Identity Management (Register & Login)

Both operations use `PreparedStatement` with placeholders (`?`), never string-concatenated SQL. The driver sends the SQL template and the parameter values to the database separately, so user-supplied text is always treated as data, never as executable SQL.

```
-- users table
CREATE TABLE users (
  user_id SERIAL PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  upi_id VARCHAR(100) UNIQUE NOT NULL,
  password VARCHAR(255) NOT NULL, -- store a hash, not plain text
  balance DECIMAL(12,2) NOT NULL DEFAULT 0.00
);
```

Logic:

- A malicious input like `"" OR '1'='1'` cannot break out of the query, because it is bound as a single `String` parameter value, not spliced into the SQL text.
- `login()` succeeds only if `rs.next()` finds a row where both `upi_id` and `password` match — a mismatch simply returns no rows rather than throwing an error, so failed logins fail gracefully.
- `try-with-resources` on the `Connection/Statement/ResultSet` guarantees they are closed even if an exception occurs, preventing connection leaks.

```
public boolean login(Connection conn, String upiId, String password) throws SQLException {
  String sql = "SELECT * FROM users WHERE upi_id = ? AND password = ?";
  try (PreparedStatement ps = conn.prepareStatement(sql)) {
    ps.setString(1, upiId);
    ps.setString(2, password); // in production: compare a hash, not plain text
    try (ResultSet rs = ps.executeQuery()) {
      return rs.next(); // true if a matching row exists
    }
  }
}

public void register(Connection conn, String name, String upiId, String password) throws
SQLException {
  String sql = "INSERT INTO users (name, upi_id, password, balance) VALUES (?, ?, ?, 0.00)";
  try (PreparedStatement ps = conn.prepareStatement(sql)) {
    ps.setString(1, name);
    ps.setString(2, upiId);
    ps.setString(3, password);
    ps.executeUpdate();
  }
}
```

QC2. Wallet Operations (Send Money) — SQL Transactions

`conn.setAutoCommit(false)` turns off the database's default behavior of committing every statement immediately, so the debit, the credit, and the ledger insert become one atomic unit of work. If any step throws (including the insufficient-balance check), `conn.rollback()` reverts every change made since the transaction began — the sender is never debited without the receiver being credited.

```
public void sendMoney(Connection conn, String senderUpi, String receiverUpi, double amount)
throws SQLException, InsufficientBalanceException {

  try {
    conn.setAutoCommit(false); // start of an all-or-nothing block

    // 1. Lock & check sender balance
    double senderBalance;
    String checkSql = "SELECT balance FROM users WHERE upi_id = ? FOR UPDATE";
```

```

try (PreparedStatement ps = conn.prepareStatement(checkSql)) {
    ps.setString(1, senderUpi);
    try (ResultSet rs = ps.executeQuery()) {
        if (!rs.next()) throw new SQLException("Sender not found");
        senderBalance = rs.getDouble("balance");
    }
}
if (amount > senderBalance) {
    throw new InsufficientBalanceException("Insufficient balance for this transfer");
}

// 2. Debit sender
try (PreparedStatement debit = conn.prepareStatement(
    "UPDATE users SET balance = balance - ? WHERE upi_id = ?")) {
    debit.setDouble(1, amount);
    debit.setString(2, senderUpi);
    debit.executeUpdate();
}

// 3. Credit receiver
try (PreparedStatement credit = conn.prepareStatement(
    "UPDATE users SET balance = balance + ? WHERE upi_id = ?")) {
    credit.setDouble(1, amount);
    credit.setString(2, receiverUpi);
    credit.executeUpdate();
}

// 4. Log the transaction
try (PreparedStatement log = conn.prepareStatement(
    "INSERT INTO transactions (sender_upi, receiver_upi, amount, status, txn_time) " +
    "VALUES (?, ?, ?, 'SUCCESS', NOW())")) {
    log.setString(1, senderUpi);
    log.setString(2, receiverUpi);
    log.setDouble(3, amount);
    log.executeUpdate();
}

conn.commit(); // all four steps succeeded -> make them permanent

} catch (SQLException | InsufficientBalanceException e) {
    conn.rollback(); // any failure -> undo every step above
    throw e;
} finally {
    conn.setAutoCommit(true); // restore default mode for the next operation
}
}

```

Logic:

- 'All-or-nothing' means partial failure states (e.g., debit succeeds but the app crashes before crediting the receiver) are impossible — `rollback()` erases the debit too.
- The balance check is done before the `UPDATE`, and (in production) `SELECT ... FOR UPDATE` row-locking prevents two simultaneous transfers from both reading a stale balance and overdrawing the account.
- `InsufficientBalanceException` is caught alongside `SQLException` so both a business-rule failure and a database failure trigger the same rollback path.
- `finally { conn.setAutoCommit(true) }` ensures the connection returns to normal auto-commit mode regardless of success or failure, so it's safe to reuse.

QC3. Transaction History

Every transfer is written once to the transactions table inside the same commit as the balance update (see C2), so the ledger and the balances can never disagree. Reading history is a simple `SELECT` filtered by either role (sender or receiver), ordered newest-first.

```
-- transactions table (permanent ledger)
CREATE TABLE transactions (
txn_id SERIAL PRIMARY KEY,
sender_upi VARCHAR(100) NOT NULL,
receiver_upi VARCHAR(100) NOT NULL,
amount DECIMAL(12,2) NOT NULL,
status VARCHAR(20) NOT NULL,
txn_time TIMESTAMP NOT NULL
);

-- Fetch a user's recent activity (sent or received)
SELECT txn_id, sender_upi, receiver_upi, amount, status, txn_time
FROM transactions
WHERE sender_upi = ? OR receiver_upi = ?
ORDER BY txn_time DESC
LIMIT 20;
```

Logic:

- WHERE sender_upi = ? OR receiver_upi = ? is required (not just sender) so a user sees money they received as well as money they sent.
- The while (rs.next()) loop is the standard JDBC pattern for turning a ResultSet into a Java collection — each iteration maps one row into one Transaction object.
- Returning a List<Transaction> (Collections Framework) lets the calling code reuse the same data for on-screen display and for file export without querying the database twice.

```
public List<Transaction> getHistory(Connection conn, String upiId) throws SQLException {
List<Transaction> history = new ArrayList<>();
String sql = "SELECT * FROM transactions WHERE sender_upi = ? OR receiver_upi = ? " +
"ORDER BY txn_time DESC LIMIT 20";
try (PreparedStatement ps = conn.prepareStatement(sql)) {
ps.setString(1, upiId);
ps.setString(2, upiId);
try (ResultSet rs = ps.executeQuery()) {
while (rs.next()) { // loop over every matching row
history.add(new Transaction(
rs.getInt("txn_id"), rs.getString("sender_upi"),
rs.getString("receiver_upi"), rs.getDouble("amount"),
rs.getString("status"), rs.getTimestamp("txn_time")));
}
}
}
return history; // an ArrayList the caller can iterate, sort, or export
}
```

QC4. File Persistence (Export Statement)

BufferedWriter wraps a FileWriter to batch character writes in memory and flush them to disk in larger chunks, which is far more efficient than writing one line at a time directly. A for-each loop over the already-fetched List<Transaction> builds one CSV line per record.

```
public void exportStatement(List<Transaction> history, String upiId, String filePath) throws
IOException {
try (BufferedWriter writer = new BufferedWriter(new FileWriter(filePath))) {
writer.write("Statement for: " + upiId);
writer.newLine();
writer.write("TxnID,Sender,Receiver,Amount,Status,Time");
writer.newLine();

for (Transaction t : history) { // for-each loop over the collection
writer.write(t.getTxnId() + "," + t.getSenderUpi() + "," + t.getReceiverUpi() + "," +
t.getAmount() + "," + t.getStatus() + "," + t.getTxnTime());
writer.newLine();
}
} // try-with-resources auto-closes and flushes the writer
System.out.println("Statement exported to " + filePath);
}
```



```
}
```

Logic:

- try-with-resources guarantees `writer.close()` runs even if an exception occurs mid-export, which also flushes any buffered but unwritten data — otherwise the last lines could be silently lost.
- Using a comma-separated header row plus one row per transaction produces a valid .csv file that can be opened directly in Excel/Sheets.
- Because `exportStatement()` takes the `List<Transaction>` already returned by `getHistory()`, no second database round-trip is needed to produce the file.